ing

PY103

Keeping it in the family: introducing inheritance

Today

Welcome to your next lesson: Keeping it in the family: introducing inheritance.

I'm preparing your personalized lesson based on what you've already learned, what's coming next, and the best way to help you grow.

Step by step, you’re moving forward.

Imagine you’ve already written a Pet class 🐾 with things like a name and an eat() method.

Now you want a Dog class. Dogs are pets, so they also have a name and can eat, but they might also have extra stuff like bark(). Inheritance is the idea that Dog can reuse what Pet already has, instead of rewriting it.

A simple picture:

Pet (parent) → Dog (subclass) → my_dog (object)

So Dog gets to use the attributes and methods defined in Pet.

Quick check for you:

If Pet defines a method get_name(), and Dog inherits from Pet, do you think a Dog object can call get_name() even if we never write get_name() inside the Dog class?

Just answer in words: “yes” or “no” — what’s your guess?

yes

Nice instinct — that “yes” is exactly where inheritance is going 🎯

Let me now show how this looks in Python with pets.

Here’s a simple parent Pet class and a subclass Dog that inherits from it:

Python

```
# Parent class
class Pet:
    """Represents a generic pet."""

    def __init__(self, name):
        self._name = name  # inherited attribute

    def get_name(self):
        """Return the pet's name."""
        return self._name

    def eat(self):
        """Make the pet eat."""
        print(self._name + " is eating.")


# Subclass
class Dog(Pet):
    """Represents a dog, which is a specific kind of Pet."""
    pass  # no extra code yet


my_dog = Dog("Rex")

print(my_dog.get_name())  # inherited method
my_dog.eat()              # inherited method
```

Key idea, step by step:

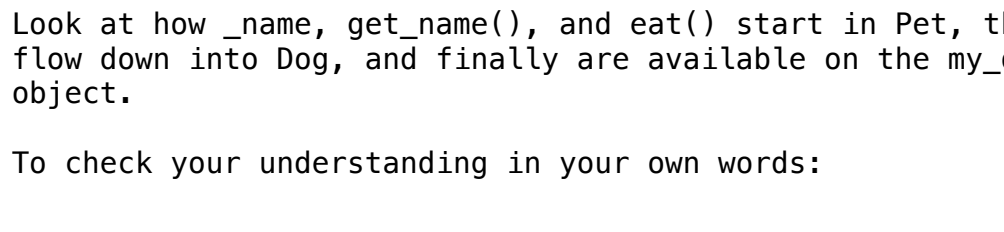
Pet defines __init__, get_name, and eat.
Dog(Pet) means: Dog inherits from Pet.

When I create Dog("Rex"), it behaves like a Pet too, so my_dog can use get_name() and eat() even though Dog itself is just pass.
Make sense so far — especially the line class Dog(Pet): meaning “Dog is a Pet and reuses its stuff”?

If yes, I’ll next show you a small visual “family tree” of Pet → Dog → my_dog.

Yes, show the family tree

Here’s the visual family tree for Pet → Dog → my_dog 👇



Look at how _name, get_name(), and eat() start in Pet, then flow down into Dog, and finally are available on the my_dog object.

To check your understanding in your own words:

Based on that diagram, what new thing (if any) did we actually write inside Dog so far — compared to Pet?

You can answer super short, like: “nothing yet” or “we added X”.

Pet to Dog Inheritance Family Tree

nothing yet just placeholder pass statement

Exactly 💯 — we didn’t add any new attributes or methods to Dog, just the pass placeholder. But my_dog still gets _name, get_name(), and eat() from Pet. That’s inheritance in action.

Let’s zoom in on one piece: inherited methods 🧠

An inherited method is a method defined in the parent class that you can call on a subclass object, without rewriting it in the subclass.

In our case:

get_name is defined in Pet
my_dog = Dog("Rex") creates a Dog object
my_dog.get_name() works, even though Dog doesn’t define get_name at all.
Quick yes/no check:

Does my_dog.eat() also count as calling an inherited method, since eat is only written in Pet?

Yes

Yes, exactly — my_dog.eat() is also using an inherited method 🐶➡️🐾

Now I want to focus on the classes themselves:

Parent class: the more general one (here: Pet)
Subclass: the more specific one that inherits from the parent (here: Dog)

In Python, the syntax is:

Python
```
class Dog(Pet):
    ...
```
You can read that as: “Dog is a Pet and inherits its behavior.”

So our mini family tree is:

Pet (parent) → Dog (subclass) → my_dog (object)

Quick check:

Looking only at this line:

Python
```
class Dog(Pet):
```
Which role does Pet play here — parent class or subclass?

Just answer: “parent” or “subclass”.

parent

Correct — Pet is the parent there 🎯

So far you’ve got:

Pet = parent class (more general)
Dog(Pet) = subclass (more specific) that inherits methods like get_name and eat
A Dog object can call those parent-defined methods without rewriting them
Now I want you to use this idea yourself.

Next step: you’ll write code in the editor.

Task (I’ll keep it small):

In the code editor, define the Pet parent class with:
__init__(self, name) that stores _name
a get_name(self) method that returns the name
Then define a Dog subclass that inherits from Pet (like we did) and just put pass inside for now.
Create a Dog object (for example Dog("Buddy")) and print(dog.get_name()).

Don’t worry about being fancy — just make it work.

Go ahead and try this in the code editor, then hit Submit code, and I’ll review it 👀

Python

```
# Parent class
class Pet:

    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name
```

```
203
204 class Dog(Pet):
205     pass
206
207
208 my_dog = Dog("Buddy")
209
210 print(my_dog.get_name())
211
212 Nice work, Ron — that’s a clean first use of inheritance ✅
213
214
215 What you just did:
216
217
218 Pet has _name and get_name().
219 Dog(Pet) means Dog inherits from Pet.
220 my_dog = Dog("Buddy") runs the parent constructor and sets
    _name.
221 my_dog.get_name() works even though Dog doesn’t define
    get_name — it’s an inherited method.
222
223 So you’ve already:
224
225
226 Created a subclass that inherits from a parent class.
227 Used inherited behavior on a subclass object. 💪
228 Quick understanding check in your own words (one short
    sentence):
229
230
231 If Dog doesn’t define get_name(), why is my_dog.get_name()
    still allowed to work?
232
233 becuase it inherited get_name frm the parernt class Pet its
    built on
234
235 Perfect explanation 🔥 Exactly right: Dog is built on Pet, so
    it inherits get_name and can use it.
236
237
238 Let’s add one more piece: inherited attributes, not just
    methods.
239
240
241 In your code, this line:
242
```

Python
def __init__(self, name):
    self._name = name
lives only in Pet. But when you do:

Python
my_dog = Dog("Buddy")
Python uses the parent __init__ to create _name for my_dog. So:

my_dog is a Dog object
It still has an inherited attribute _name coming from Pet
That’s why get_name() has something to return

So _name is an inherited attribute.

Quick check:

For the my_dog object, is _name coming from:

the Dog class itself, or
the Pet constructor it inherited?

Answer in a short phrase.

the pet class contructer

Exactly — _name comes from the Pet constructor and is inherited by Dog 🐾

You’ve now seen:

Parent class: Pet
Subclass: Dog(Pet)
Inherited attributes: _name
Inherited methods: get_name(), and we also had eat() earlier
Let’s do one more small hands-on step: add a new method to Pet and use it from Dog.

In the editor, starting from your working code:

In Pet, add a new method:
Call it eat(self)
Make it print(self._name + " is eating.")
Don’t touch Dog (keep it just inheriting from Pet).
After print(my_dog.get_name()), also call my_dog.eat().

Run and Submit code, and I’ll check that your Dog is using the new inherited method 🐶

Python

```
# Parent class
class Pet:

    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    # Perhaps canibilize is more fitting
    def eat(self):
        print(self._name + " is eating.")

class Dog(Pet):
    pass


my_dog = Dog("Buddy")

print(my_dog.get_name())
my_dog.eat()
```

This is great, Ron 👌

What your code shows clearly:

Pet defines _name, get_name(), and eat().
Dog(Pet) inherits all of that without adding anything.
my_dog = Dog("Buddy") ends up with:
an inherited attribute _name
inherited methods get_name() and eat()
my_dog.eat() is calling a parent-defined method on a subclass object — exactly what I wanted you to see.

So at this point you’ve:

Understood what inheritance is for (reusing behavior).
Created a subclass that inherits from a parent.
Used inherited behavior on a subclass object. 🎯
Now I want to quickly check your understanding with a small multiple-choice question about this Pet/Dog setup.

Ready for a quick check? Just say “yes” and I’ll show it.

Yes, I'm ready

What do you think? 👇

In our Pet / Dog example, my_dog is created from the Dog class. Which surprise power does my_dog get from its parent class Pet?

A.
It can use methods defined in Pet.

B.
It automatically becomes a list.

C.
It must use super().

D.
It can override every method automatically.

Nice, you nailed that one 🎯

You chose: “It can use methods defined in Pet.”

That’s exactly what inheritance gives my_dog:

Dog inherits from Pet
So a Dog object can call all methods defined in Pet (like get_name() and eat()) without rewriting them.

The other options are off because:

It doesn’t become a list — it’s still a Dog object.
It doesn’t have to use super() in our example at all.
It doesn’t automatically override anything; it just inherits.
Before we move toward wrapping this lesson, I’d love to have you try one more small inheritance exercise in a different setting (not pets).

Up for a short practice like that, or do you feel done with inheritance for now?

Let's try a practice

Love it 🌟 Let’s switch domains and keep it simple.

Step by step, you’ll mirror what you did with Pet / Dog, but now with vehicles.

Here’s the idea:

Parent class: Vehicle
stores a _brand
has a method get_brand() that returns the brand
Subclass: Bicycle(Vehicle)
doesn’t add anything new yet (just like Dog at first)
Object: my_bike = Bicycle("Trek")
can call my_bike.get_brand() even though Bicycle doesn’t define it
Your task in the code editor 🧠💻:

Define a Vehicle class:
__init__(self, brand) that saves self._brand = brand
get_brand(self) that returns self._brand
Define a Bicycle class that inherits from Vehicle and just uses pass inside.
Create a Bicycle object (e.g. my_bike = Bicycle("Trek")).
print(my_bike.get_brand()).

When you’re done, hit Submit code, and I’ll check how your

inheritance tree looks 🚲

Python

```
# Parent class
class Vehicle:

    def __init__(self, brand):
        self._brand = brand

    def get_brand(self):
        return self._brand


class Bicycle(Vehicle):
    pass


my_bike = Bicycle("Trek")
print(my_bike.get_brand())
```

Beautifully done, Ron 🎉

In this lesson you:

Grasped inheritance as “reuse what the parent class already has.”
Defined clear parent classes (Pet, Vehicle) and subclasses (Dog, Bicycle) using class Subclass(Parent):.
Created subclass objects (my_dog, my_bike) that:
used inherited attributes (_name, _brand), and
called inherited methods (get_name(), eat(), get_brand()).

Your Bicycle example is spot on:

```
class Vehicle:
    def __init__(self, brand):
        self._brand = brand

    def get_brand(self):
        return self._brand

class Bicycle(Vehicle):
    pass

my_bike = Bicycle("Trek")
```

```
print(my_bike.get_brand())

This shows exactly the pattern:


Vehicle (parent) → Bicycle (subclass) → my_bike (object using inherited methods)



If you want, I can:


compare inheritance vs. just copying code, or
give you a quick mini-challenge mixing Pet, Dog, and another subclass (like Cat) 🐱

Or you can jump straight to the next lesson and keep the momentum going. 🚀




# Parent class
class Pet:

    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    # Perhaps canibilize is more fitting
    def eat(self):
        print(self._name + " is eating.")

class Dog(Pet):
    pass


my_dog = Dog("Buddy")

print(my_dog.get_name())
my_dog.eat()

# Parent class
class Vehicle:
```

```

    def __init__(self, brand):
        self._brand = brand

    def get_brand(self):
        return self._brand


class Bicycle(Vehicle):
    pass


my_bike = Bicycle("Trek")
print(my_bike.get_brand())

```